Architecture Concept Guide

作成日: 2026年10月 / 対象: 新規参画エンジニア

アーキテクチャ設計思想と開発の考え方

～「自己防衛するドメイン」と4層の厳格な境界線～

本プロジェクト（webapp2）は、Spring Boot等の重量級フレームワークに依存せず、純粋なJavaとServlet/JSP環境下で**「クリーンアーキテクチャ（Clean Architecture / Hexagonal）」**の原則を徹底して実装しています。
なぜこの設計思想を採用しているのか、何を守らなければならないのかを解説します。

Chapter 01

コアメッセージ：DBや画面は「詳細」にすぎない

多くの業務システム開発では、「テーブルを設計し、画面を作り、その間にSQLとgetter/setterを書く」というアプローチが採られがちです。 しかし、SES勤怠管理のような複雑なビジネスルール（精算幅、36協定アラート、中抜け休憩、週報多層決裁）を持つシステムでそれをやると、 **業務ルールがController、DAO、JSPのあちこちに散らばり、少しの変更でバグが多発する（デグレの泥沼）**に陥ります。

❌ 従来のアンチパターン（DB中心・画面中心）

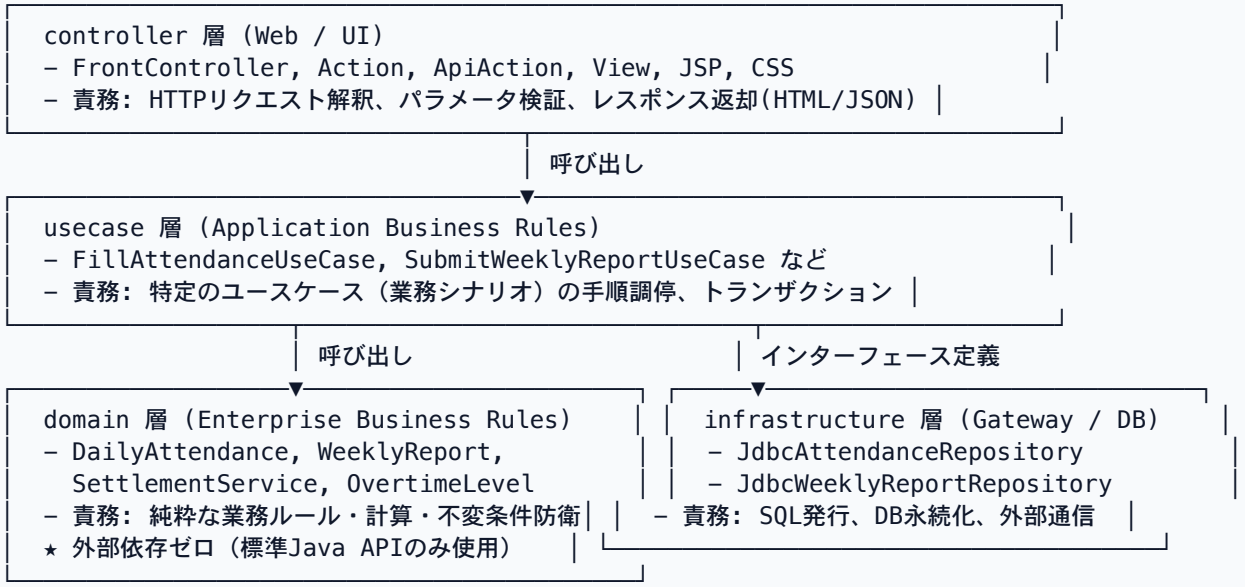
- DBの都合（カラム型やNULL可否）にドメインロジックが引きずられる
- TomcatやMySQLが動いていないと単体テストができない
- Beanはただの箱（getter/setter）で、不正な値でも代入できてしまう

⭕ 本プロジェクトの設計思想（ビジネスルール中心）

- ドメイン層は標準Javaのみ。ServletもJDBCも知らない（外部依存ゼロ）
- DB起動不要で、全テスト（20件）が0.8秒で完了する
- オブジェクト自身が不正な状態を拒絶する（自己防衛モデル）

Chapter 02

4層クリーンアーキテクチャの責務と境界線



鉄則：依存性の方向（Dependency Rule）

依存の矢印は必ず**「外側から内側へ」**向かいます。内側にある domain 層は、外側の infrastructure（SQL/JDBC）や controller（Servlet/JSP）を絶対に import してはいけません。

Chapter 03

「貧血ドメインモデル」の根絶と自己防衛

本プロジェクトのエンティティには、publicな setter を原則として持たせません。 データ変更は必ず**「業務意味のあるメソッド」**を経由し、不正な状態が発生した瞬間に例外を投げます。

```
// domain.attendance.DailyAttendance の自己防衛コード例
public void fillAttendance(LocalTime startTime, LocalTime endTime, int breakMinutes,
                           int abstractId, LeaveType leaveType, String remarks, List<Work
    // 1. 月次確定ロックの防衛
    if (this.isLocked) {
        throw new IllegalStateException("月次確定によりロックされているため変更できません。");
    }
    this.startTime = startTime;
    this.endTime = endTime;
    this.breakMinutes = breakMinutes;
    this.segments.clear();
    if (newSegments != null) this.segments.addAll(newSegments);

    // 2. 実労働時間と内訳セグメント（客先＋社内）の整合性を自己検証！
    validateSegmentsIntegrity();
}

private void validateSegmentsIntegrity() {
    int total = calculateTotalWorkMinutes();
    int segmentsTotal = getDirectMinutes() + getIndirectMinutes();
    if (total != segmentsTotal) {
        throw new IllegalArgumentException(
            String.format("実労働時間(%d分)と内訳セグメント合計(%d分)が一致しません。", total, seg
        );
    }
}
```

💡 **これにより得られる効果:** メモリ上に「時間が合わない不正な勤怠データ」や「ロックされているのに書き換えられたデータ」が**1ミリ秒たりとも存在できなくなります**。バグは即座に発見され、DBまで汚染されません。

Chapter 04

新規機能・改修時の思考ステップ（どこに何を書くか？）

書きたいロジックの内容	配置する層	実装クラス例
「精算幅の超過・不足額を計算したい」「残業が45h超えたか判定したい」	domain/	SettlementService, Article36ComplianceService
「週報を提出する（未入力チェック → 提出状態変更 → メール通知準備）」	usecase/	SubmitWeeklyReportUseCase
「MySQLから月間の勤怠一覧をSELECTしたい」「UPDATE文を実行したい」	infrastructure/	JdbcAttendanceRepository, ConnectionBase
「リクエストからyearMonthを受け取る」「JSONを返却する」「JSPに渡す」	controller/	SesAttendanceAction, SesAttendanceApiAction